

中国海洋大学计算机科学与技术学院

实验三 bomb lab 实验报告

姓名：庄环嘉 年级：2022 级 专业：计算机科学与技术

科目：计算机系统基础 题目：bomb lab

实验时间：2025 年 1 月 3 日 实验教师：蒋永国

一、实验目的：

熟悉和掌握汇编语言及调试。

二、实验平台：

Educoder 在线实践教学平台及 Linux

三、实验任务及要求：

逆向工程拆除“二进制炸弹”程序增强对程序机器级表示、汇编语言、调试器和逆向工程等理解。一个“Binary Bombs”（二进制炸弹，简称炸弹）是一个 Linux 可执行 C 程序，包含 phase1~phase6 共 6 个阶段。炸弹运行各阶段要求输入一个字符串，若输入符合程序预期，该阶段炸弹被“拆除”，否则“爆炸”。你需要拆除尽可能多的炸弹
具体题目见头歌网站

四、实验前的准备：

1. 通过 ssh 远程登录 Linux 服务器

```
zhj@zhj-virtual-machine:~$ ssh -p 52901 root@47.96.157.89
The authenticity of host '[47.96.157.89]:52901 ([47.96.157.89]:52901)' can't be
established.
ED25519 key fingerprint is SHA256:LRHx5wp5Vjake0uQ2VnUi9dWSE+mAYuiM2wviYrZDgE.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[47.96.157.89]:52901' (ED25519) to the list of known
hosts.
root@47.96.157.89's password:
版本库代码路径：/data/workspace/myshixun/
```

2. 在命令行执行以下命令来找到对应 bomb 文件：cd /data/workspace/myshixun/step1

3. 反汇编找出源码：objdump -d ./bomb > bomb.s

4. 使用 ls 命令查看该文件夹下的文件，chmod 777 bomb 来修改运行权限

5. gdb bomb 进入调试

```

root@educoder:~# cd /data/workspace/myshixun/step1
root@educoder:/data/workspace/myshixun/step1# objdump -d bomb > bomb.s
root@educoder:/data/workspace/myshixun/step1# objdump -d ./bomb > bomb.s
root@educoder:/data/workspace/myshixun/step1# ls
bomb  bomb.s  main.c  makefile  test.sh  test.txt
root@educoder:/data/workspace/myshixun/step1# chmod 777 bomb
root@educoder:/data/workspace/myshixun/step1# ls
bomb  bomb.s  main.c  makefile  test.sh  test.txt

```

五、 实验过程及结果:

1. Phase1

● 实验代码:

```

Dump of assembler code for function phase_1:
0x0000000000400ee0 <+0>:      sub    $0x8,%rsp
0x0000000000400ee4 <+4>:      mov    $0x402400,%esi
0x0000000000400ee9 <+9>:      callq 0x401338 <strings_not_equal>
0x0000000000400eee <+14>:     test   %eax,%eax
0x0000000000400ef0 <+16>:     je     0x400ef7 <phase_1+23>
0x0000000000400ef2 <+18>:     callq 0x40143a <explode_bomb>
0x0000000000400ef7 <+23>:     add    $0x8,%rsp
0x0000000000400efb <+27>:     retq

```

● 实验过程:

1. 对代码进行注释。

1. 0x0000000000400ee0 <+0>: sub \$0x8,%rsp # 在栈中开辟一个8字节的临时空间
2. 0x0000000000400ee4 <+4>: mov \$0x402400,%esi # 将0x402400的值作为
<string not equal>的参数传入
3. 0x0000000000400ee9 <+9>: callq 0x401338 <strings_not_equal>
4. 0x0000000000400eee <+14>: test %eax,%eax # 测试该函数返回值
5. 0x0000000000400ef0 <+16>: je 0x400ef7 <phase_1+23> # 若返回值为0则跳过炸
弹爆炸函数
6. 0x0000000000400ef2 <+18>: callq 0x40143a <explode_bomb> # 返回值为1, 炸
7. 0x0000000000400ef7 <+23>: add \$0x8,%rsp # 恢复栈
8. 0x0000000000400efb <+27>: etq

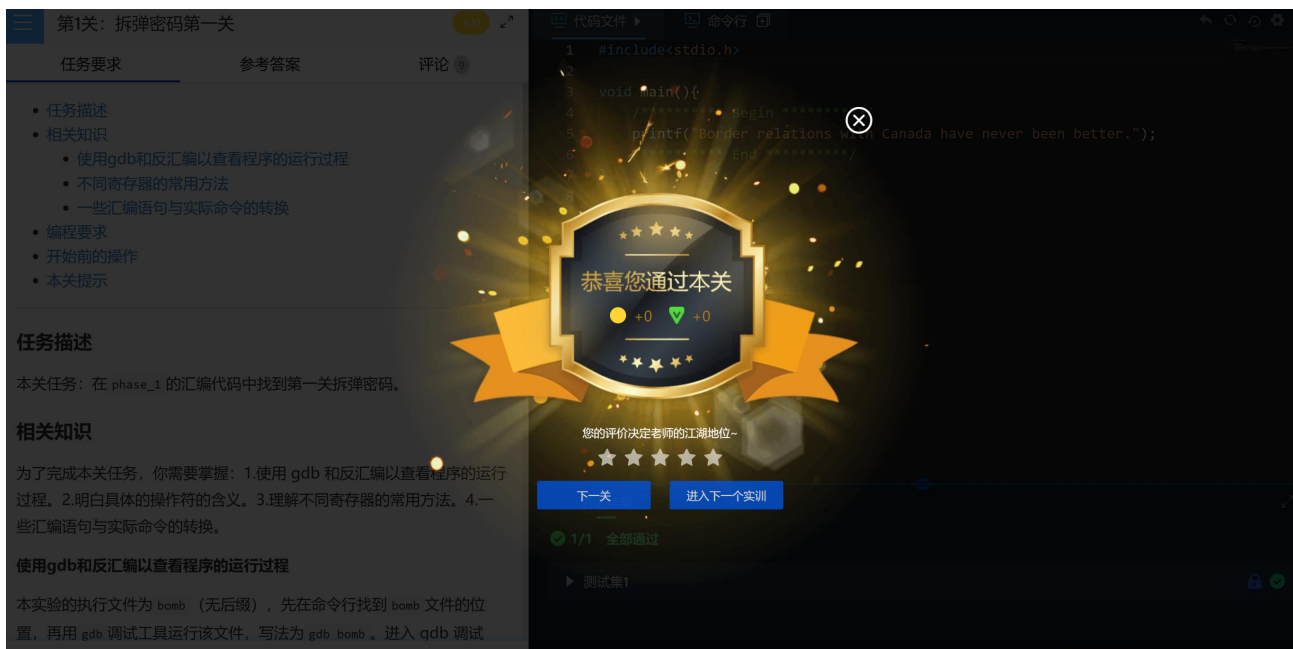
2. 使用 x/s 命令将 0x402400 中存储的字符串导出, 得出答案。

```

(gdb) x/s 0x402400
0x402400:      "Border relations with Canada have never been better."

```

● 实验结果:



2. Phase2

● 实验代码：

Dump of assembler code for function phase_2:

```
0x0000000000400efc <+0>:      push    %rbp
0x0000000000400efd <+1>:      push    %rbx
0x0000000000400efe <+2>:      sub     $0x28,%rsp
0x0000000000400f02 <+6>:      mov     %rsp,%rsi
0x0000000000400f05 <+9>:      callq   0x40145c <read_six_numbers>
0x0000000000400f0a <+14>:     cmpl    $0x1,(%rsp)
0x0000000000400f0e <+18>:     je      0x400f30 <phase_2+52>
0x0000000000400f10 <+20>:     callq   0x40143a <explode_bomb>
0x0000000000400f15 <+25>:     jmp     0x400f30 <phase_2+52>
0x0000000000400f17 <+27>:     mov     -0x4(%rbx),%eax
0x0000000000400f1a <+30>:     add     %eax,%eax
0x0000000000400f1c <+32>:     cmp     %eax,(%rbx)
0x0000000000400f1e <+34>:     je      0x400f25 <phase_2+41>
0x0000000000400f20 <+36>:     callq   0x40143a <explode_bomb>
0x0000000000400f25 <+41>:     add     $0x4,%rbx
0x0000000000400f29 <+45>:     cmp     %rbp,%rbx
0x0000000000400f2c <+48>:     jne     0x400f17 <phase_2+27>
0x0000000000400f2e <+50>:     jmp     0x400f3c <phase_2+64>
0x0000000000400f30 <+52>:     lea     0x4(%rsp),%rbx
0x0000000000400f35 <+57>:     lea     0x18(%rsp),%rbp
0x0000000000400f3a <+62>:     jmp     0x400f17 <phase_2+27>
0x0000000000400f3c <+64>:     add     $0x28,%rsp
```



```

---Type <return> to continue, or q <return> to quit---
0x0000000000400f40 <+68>:    pop     %rbx
0x0000000000400f41 <+69>:    pop     %rbp
0x0000000000400f42 <+70>:    retq

```

● 实验过程：

1. 首先函数将栈指针传入参数寄存器，猜测栈指针作为参数用于保存数字， %rdi 给出输入字符串地址。因此，函数 read_six_number 函数用于将输入的字符串转换为 6 个数字。得知，需要输入 6 个数字作为密钥。

2. 第一个输入的数字是 1。

由

1. <+41>（偏移量++）
2. <+57>（尾指针的赋值）
3. <+45>（偏移量等于尾值）

判断出，这函数当中存在一个循环。循环中 %rbx 依次保存栈中的所有元素的地址。而又由 <+27>-<+32>语句中可知，栈中元素满足这样的排列：栈中每一元素是它上一元素的两倍

3. 得到最后答案为：“1 2 4 8 16 32”。

● 实验结果：

第2关：拆弹密码第二关

任务要求 参考答案 评论

- 任务描述
- 相关知识
 - 使用gdb和反汇编以查看程序的运行过程
 - 不同寄存器的常用方法
 - 一些汇编语句与实际命令的转换
- 编程要求
- 开始前的操作
- 本关提示

任务描述

本关任务：在 phase_2 的汇编代码中找到本关拆弹密码。

相关知识

为了完成本关任务，你需要掌握：

- 使用 gdb 和反汇编以查看程序的运行过程。
- 明白具体的操作符的含义。
- 理解不同寄存器的常用方法。
- 一些汇编语句与实际命令的转换。

使用gdb和反汇编以查看程序的运行过程

恭喜您通过本关

您的评价决定老师的江湖地位~

下一关 进入下一个实训

1/1 全部通过

测试集1

3. Phase3

● 实验代码：

Dump of assembler code for function phase_3:

```
0x0000000000400f43 <+0>:      sub     $0x18,%rsp
0x0000000000400f47 <+4>:      lea     0xc(%rsp),%rcx
0x0000000000400f4c <+9>:      lea     0x8(%rsp),%rdx
0x0000000000400f51 <+14>:     mov     $0x4025cf,%esi
0x0000000000400f56 <+19>:     mov     $0x0,%eax
0x0000000000400f5b <+24>:     callq  0x400bf0 <__isoc99_sscanf@plt>
0x0000000000400f60 <+29>:     cmp     $0x1,%eax
0x0000000000400f63 <+32>:     jg      0x400f6a <phase_3+39>
0x0000000000400f65 <+34>:     callq  0x40143a <explode_bomb>
0x0000000000400f6a <+39>:     cmpl    $0x7,0x8(%rsp)
0x0000000000400f6f <+44>:     ja      0x400fad <phase_3+106>
0x0000000000400f71 <+46>:     mov     0x8(%rsp),%eax
0x0000000000400f75 <+50>:     jmpq    *0x402470(,%rax,8)
0x0000000000400f7c <+57>:     mov     $0xcf,%eax
0x0000000000400f81 <+62>:     jmp     0x400fbe <phase_3+123>
0x0000000000400f83 <+64>:     mov     $0x2c3,%eax
0x0000000000400f88 <+69>:     jmp     0x400fbe <phase_3+123>
0x0000000000400f8a <+71>:     mov     $0x100,%eax
0x0000000000400f8f <+76>:     jmp     0x400fbe <phase_3+123>
0x0000000000400f91 <+78>:     mov     $0x185,%eax
0x0000000000400f96 <+83>:     jmp     0x400fbe <phase_3+123>
0x0000000000400f98 <+85>:     mov     $0xce,%eax
```

```
0x0000000000400f9d <+90>:     jmp     0x400fbe <phase_3+123>
0x0000000000400f9f <+92>:     mov     $0x2aa,%eax
0x0000000000400fa4 <+97>:     jmp     0x400fbe <phase_3+123>
0x0000000000400fa6 <+99>:     mov     $0x147,%eax
0x0000000000400fab <+104>:    jmp     0x400fbe <phase_3+123>
0x0000000000400fad <+106>:    callq  0x40143a <explode_bomb>
0x0000000000400fb2 <+111>:    mov     $0x0,%eax
0x0000000000400fb7 <+116>:    jmp     0x400fbe <phase_3+123>
0x0000000000400fb9 <+118>:    mov     $0x137,%eax
0x0000000000400fbe <+123>:    cmp     0xc(%rsp),%eax
0x0000000000400fc2 <+127>:    je      0x400fc9 <phase_3+134>
0x0000000000400fc4 <+129>:    callq  0x40143a <explode_bomb>
0x0000000000400fc9 <+134>:    add     $0x18,%rsp
0x0000000000400fcd <+138>:    retq
```

● 实验过程:

1. 使用 x/s 命令查看, 得知是 “%d %d”, 即需要输入两个整数。两个参数, 分别是栈的+12 地址, 栈的+8 地址。

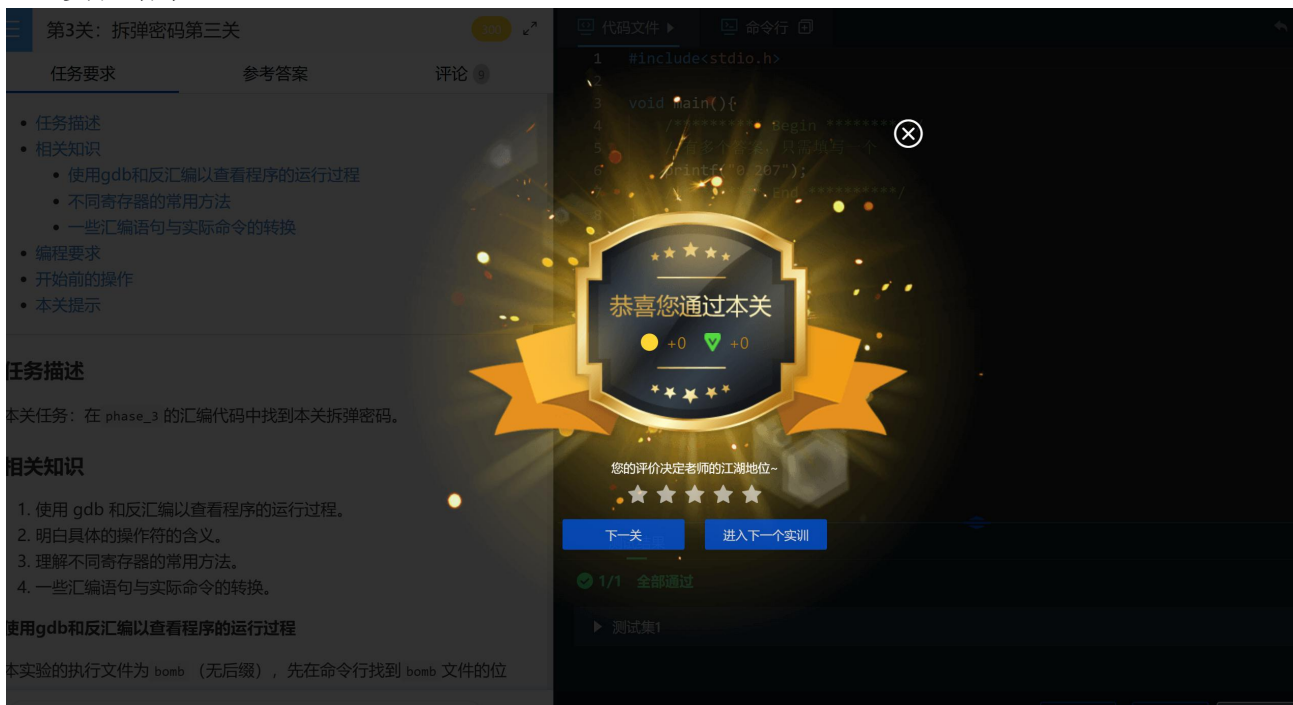
```
(gdb) x/s 0x4025cf
0x4025cf:      "%d %d"
```

2. 在<+44>中，将第一个数字与 7 进行无符号的小于比较。这是在规定输入的第一个数字必须是 0-6 之间的第一个数。第二个数字通过 switch-case 语句对应 0-6，需输入不同数字。

```
(gdb) x/16a 0x402470
0x402470:      0x400f7c <phase_3+57>  0x400fb9 <phase_3+118>
0x402480:      0x400f83 <phase_3+64>  0x400f8a <phase_3+71>
0x402490:      0x400f91 <phase_3+78>  0x400f98 <phase_3+85>
0x4024a0:      0x400f9f <phase_3+92>  0x400fa6 <phase_3+99>
```

3. 得到最后答案为：“0 207”。

● 实验结果：



4. Phase4

● 实验代码：

Dump of assembler code for function phase_4:

```
0x000000000040100c <+0>:      sub     $0x18,%rsp
0x0000000000401010 <+4>:      lea     0xc(%rsp),%rcx
0x0000000000401015 <+9>:      lea     0x8(%rsp),%rdx
0x000000000040101a <+14>:     mov     $0x4025cf,%esi
0x000000000040101f <+19>:     mov     $0x0,%eax
0x0000000000401024 <+24>:     callq  0x400bf0 <__isoc99_sscanf@plt>
0x0000000000401029 <+29>:     cmp     $0x2,%eax
0x000000000040102c <+32>:     jne     0x401035 <phase_4+41>
0x000000000040102e <+34>:     cmpl    $0xe,0x8(%rsp)
0x0000000000401033 <+39>:     jbe     0x40103a <phase_4+46>
0x0000000000401035 <+41>:     callq  0x40143a <explode_bomb>
0x000000000040103a <+46>:     mov     $0xe,%edx
0x000000000040103f <+51>:     mov     $0x0,%esi
0x0000000000401044 <+56>:     mov     0x8(%rsp),%edi
0x0000000000401048 <+60>:     callq  0x400fce <func4>
0x000000000040104d <+65>:     test    %eax,%eax
0x000000000040104f <+67>:     jne     0x401058 <phase_4+76>
0x0000000000401051 <+69>:     cmpl    $0x0,0xc(%rsp)
0x0000000000401056 <+74>:     je      0x40105d <phase_4+81>
0x0000000000401058 <+76>:     callq  0x40143a <explode_bomb>
0x000000000040105d <+81>:     add     $0x18,%rsp
0x0000000000401061 <+85>:     retq
```

● 实验过程:

1. 通过 disas 命令获取汇编代码, 注释如下:

```
1.      0x000000000040100c <+0>: sub     $0x18,%rsp
2.      0x0000000000401010 <+4>: lea     0xc(%rsp),%rcx    //要用的参数, 放入参数寄存器
    中给 scanf 存
3.      0x0000000000401015 <+9>: lea     0x8(%rsp),%rdx    //要用的参数
4.      0x000000000040101a <+14>: mov     $0x4025cf,%esi    //这个也是"%d %d"
5.      0x000000000040101f <+19>: mov     $0x0,%eax
6.      0x0000000000401024 <+24>: callq  0x400bf0 <__isoc99_sscanf@plt>
7.      0x0000000000401029 <+29>: cmp     $0x2,%eax    //不是正常的两个参数就炸
8.      0x000000000040102c <+32>: jne     0x401035 <phase_4+41>
9.      0x000000000040102e <+34>: cmpl    $0xe,0x8(%rsp) //这个值和 14
10.     0x0000000000401033 <+39>: jbe     0x40103a <phase_4+46> //低于或者相等
11.     0x0000000000401035 <+41>: callq  0x40143a <explode_bomb>
12.     0x000000000040103a <+46>: mov     $0xe,%edx
13.     0x000000000040103f <+51>: mov     $0x0,%esi
14.     0x0000000000401044 <+56>: mov     0x8(%rsp),%edi
```

```

15.      0x0000000000401048 <+60>: callq  0x400fce <func4>
16.      0x000000000040104d <+65>: test   %eax,%eax  //返回 0 才是正确做法
17.      0x000000000040104f <+67>: jne    0x401058 <phase_4+76>
18.      0x0000000000401051 <+69>: cmpl   $0x0,0xc(%rsp)  //再比较第二个输入值和 0 的

```

关系

```

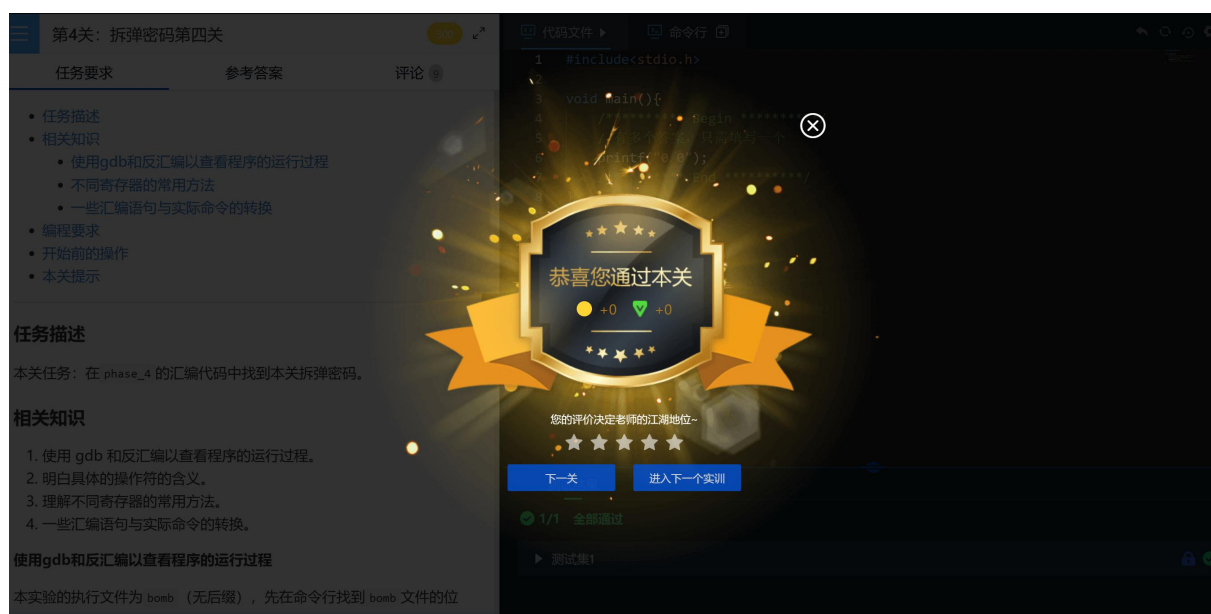
19.      0x0000000000401056 <+74>: je     0x40105d <phase_4+81>  //需等于 0
20.      0x0000000000401058 <+76>: callq  0x40143a <explode_bomb>
21.      0x000000000040105d <+81>: add     $0x18,%rsp
22.      0x0000000000401061 <+85>: retq

```

2. 当返回值为 0 时，第一个数字需为 7。<+69>语句规定了第二个数字需为 0。

3. 得到最后答案为：“7 0”。

● 实验结果：



5. Phase5

● 实验代码：


```

0x000000000004010b3 <+81>:    mov     $0x40245e,%esi
---Type <return> to continue, or q <return> to quit---
0x000000000004010b8 <+86>:    lea     0x10(%rsp),%rdi
0x000000000004010bd <+91>:    callq  0x401338 <strings_not_equal>
0x000000000004010c2 <+96>:    test    %eax,%eax
0x000000000004010c4 <+98>:    je      0x4010d9 <phase_5+119>
0x000000000004010c6 <+100>:   callq  0x40143a <explode_bomb>
0x000000000004010cb <+105>:   nopl    0x0(%rax,%rax,1)
0x000000000004010d0 <+110>:   jmp     0x4010d9 <phase_5+119>
0x000000000004010d2 <+112>:   mov     $0x0,%eax
0x000000000004010d7 <+117>:   jmp     0x40108b <phase_5+41>
0x000000000004010d9 <+119>:   mov     0x18(%rsp),%rax
0x000000000004010de <+124>:   xor     %fs:0x28,%rax
0x000000000004010e7 <+133>:   je      0x4010ee <phase_5+140>
0x000000000004010e9 <+135>:   callq  0x400b30 <__stack_chk_fail@plt>
0x000000000004010ee <+140>:   add     $0x20,%rsp
0x000000000004010f2 <+144>:   pop     %rbx
0x000000000004010f3 <+145>:   retq

```

```

0x00000000000401062 <+0>:    push    %rbx
0x00000000000401063 <+1>:    sub     $0x20,%rsp
0x00000000000401067 <+5>:    mov     %rdi,%rbx
0x0000000000040106a <+8>:    mov     %fs:0x28,%rax
0x00000000000401073 <+17>:   mov     %rax,0x18(%rsp)
0x00000000000401078 <+22>:   xor     %eax,%eax
0x0000000000040107a <+24>:   callq  0x40131b <string_length>
0x0000000000040107f <+29>:   cmp     $0x6,%eax
0x00000000000401082 <+32>:   je      0x4010d2 <phase_5+112>
0x00000000000401084 <+34>:   callq  0x40143a <explode_bomb>
0x00000000000401089 <+39>:   jmp     0x4010d2 <phase_5+112>
0x0000000000040108b <+41>:   movzbl  (%rbx,%rax,1),%ecx
0x0000000000040108f <+45>:   mov     %cl,(%rsp)
0x00000000000401092 <+48>:   mov     (%rsp),%rdx
0x00000000000401096 <+52>:   and     $0xf,%edx
0x00000000000401099 <+55>:   movzbl  0x4024b0(%rdx),%edx
0x000000000004010a0 <+62>:   mov     %dl,0x10(%rsp,%rax,1)
0x000000000004010a4 <+66>:   add     $0x1,%rax
0x000000000004010a8 <+70>:   cmp     $0x6,%rax
0x000000000004010ac <+74>:   jne     0x40108b <phase_5+41>
0x000000000004010ae <+76>:   movb    $0x0,0x16(%rsp)
0x000000000004010b3 <+81>:   mov     $0x40245e,%esi

```

- 实验过程:

- 实验结果:

1. 根据<+29>语句, <string_length>函数的返回值, 即输入字符串的长度必须等于 6。

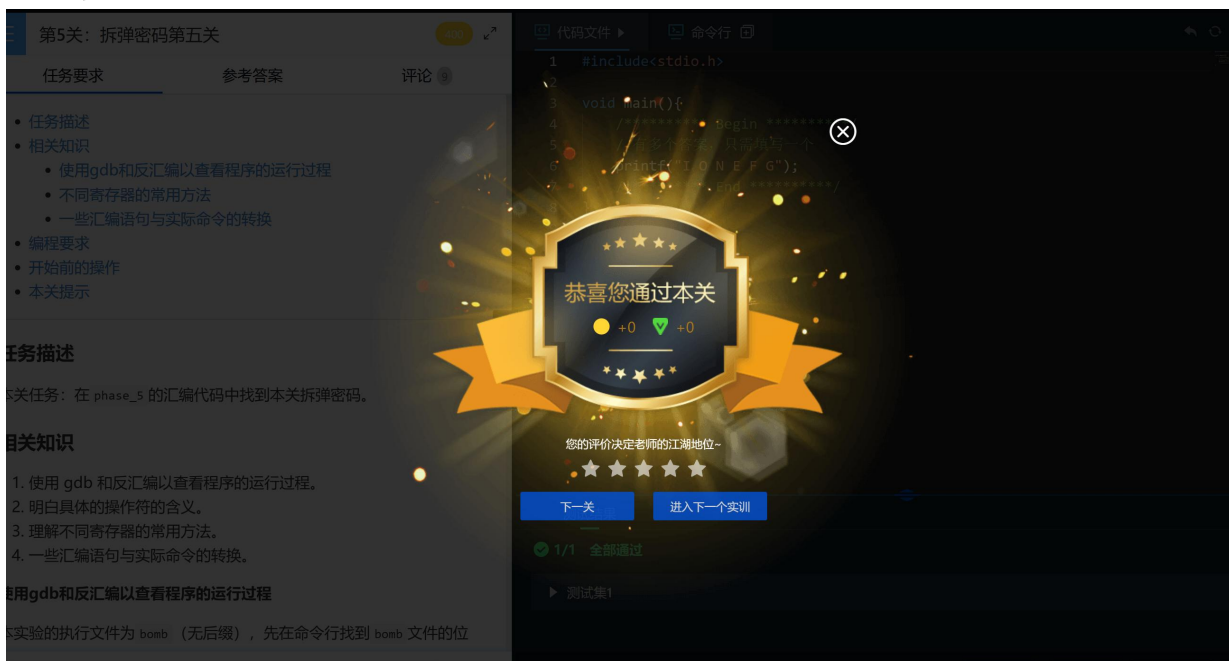
2. <+55>语句得输入的字符作为偏移量, 可以刚好使得 0x4024b0 作为首地址偏移所对应得新的字符与 0x40245e 对应得字符依次相等, 使用 x/s 0x4024b0 以及 x/s 0x40245e 命令查看两个字符串。

```
(gdb) x/s 0x4024b0
0x4024b0 <array.3449>: "maduiersnfotvbylSo you think you can stop the bomb with ctrl-c, do you?"
```

```
(gdb) x/s 0x40245e
0x40245e: "flyers"
```

得到, 偏移量应该为 9 15 14 5 6 7。

3. 故最后答案为: “9 15 14 5 6 7”。



6. Phase6

- 实验代码:

```

0x00000000004010f4 <+0>:    push    %r14
0x00000000004010f6 <+2>:    push    %r13
0x00000000004010f8 <+4>:    push    %r12
0x00000000004010fa <+6>:    push    %rbp
0x00000000004010fb <+7>:    push    %rbx
0x00000000004010fc <+8>:    sub     $0x50,%rsp
0x0000000000401100 <+12>:   mov     %rsp,%r13
0x0000000000401103 <+15>:   mov     %rsp,%rsi
0x0000000000401106 <+18>:   callq   0x40145c <read_six_numbers>
0x000000000040110b <+23>:   mov     %rsp,%r14
0x000000000040110e <+26>:   mov     $0x0,%r12d
0x0000000000401114 <+32>:   mov     %r13,%rbp
0x0000000000401117 <+35>:   mov     0x0(%r13),%eax
0x000000000040111b <+39>:   sub     $0x1,%eax
0x000000000040111e <+42>:   cmp     $0x5,%eax
0x0000000000401121 <+45>:   jbe     0x401128 <phase_6+52>
0x0000000000401123 <+47>:   callq   0x40143a <explode_bomb>
0x0000000000401128 <+52>:   add     $0x1,%r12d
0x000000000040112c <+56>:   cmp     $0x6,%r12d
0x0000000000401130 <+60>:   je      0x401153 <phase_6+95>
0x0000000000401132 <+62>:   mov     %r12d,%ebx
0x0000000000401135 <+65>:   movslq  %ebx,%rax

```



```

0x0000000000401135 <+65>:    movslq %ebx,%rax
--Type <return> to continue, or q <return> to quit--
0x0000000000401138 <+68>:    mov     (%rsp,%rax,4),%eax
0x000000000040113b <+71>:    cmp     %eax,0x0(%rbp)
0x000000000040113e <+74>:    jne     0x401145 <phase_6+81>
0x0000000000401140 <+76>:    callq   0x40143a <explode_bomb>
0x0000000000401145 <+81>:    add     $0x1,%ebx
0x0000000000401148 <+84>:    cmp     $0x5,%ebx
0x000000000040114b <+87>:    jle     0x401135 <phase_6+65>
0x000000000040114d <+89>:    add     $0x4,%r13
0x0000000000401151 <+93>:    jmp     0x401114 <phase_6+32>
0x0000000000401153 <+95>:    lea     0x18(%rsp),%rsi
0x0000000000401158 <+100>:   mov     %r14,%rax
0x000000000040115b <+103>:   mov     $0x7,%ecx
0x0000000000401160 <+108>:   mov     %ecx,%edx
0x0000000000401162 <+110>:   sub     (%rax),%edx
0x0000000000401164 <+112>:   mov     %edx,(%rax)
0x0000000000401166 <+114>:   add     $0x4,%rax
0x000000000040116a <+118>:   cmp     %rsi,%rax
0x000000000040116d <+121>:   jne     0x401160 <phase_6+108>
0x000000000040116f <+123>:   mov     $0x0,%esi
0x0000000000401174 <+128>:   jmp     0x401197 <phase_6+163>
0x0000000000401176 <+130>:   mov     0x8(%rdx),%rdx
0x000000000040117a <+134>:   add     $0x1,%eax

```

```

0x000000000040117d <+137>:   cmp     %ecx,%eax
--Type <return> to continue, or q <return> to quit--
0x000000000040117f <+139>:   jne     0x401176 <phase_6+130>
0x0000000000401181 <+141>:   jmp     0x401188 <phase_6+148>
0x0000000000401183 <+143>:   mov     $0x6032d0,%edx
0x0000000000401188 <+148>:   mov     %rdx,0x20(%rsp,%rsi,2)
0x000000000040118d <+153>:   add     $0x4,%rsi
0x0000000000401191 <+157>:   cmp     $0x18,%rsi
0x0000000000401195 <+161>:   je      0x4011ab <phase_6+183>
0x0000000000401197 <+163>:   mov     (%rsp,%rsi,1),%ecx
0x000000000040119a <+166>:   cmp     $0x1,%ecx
0x000000000040119d <+169>:   jle     0x401183 <phase_6+143>
0x000000000040119f <+171>:   mov     $0x1,%eax
0x00000000004011a4 <+176>:   mov     $0x6032d0,%edx
0x00000000004011a9 <+181>:   jmp     0x401176 <phase_6+130>
0x00000000004011ab <+183>:   mov     0x20(%rsp),%rbx
0x00000000004011b0 <+188>:   lea     0x28(%rsp),%rax
0x00000000004011b5 <+193>:   lea     0x50(%rsp),%rsi
0x00000000004011ba <+198>:   mov     %rbx,%rcx
0x00000000004011bd <+201>:   mov     (%rax),%rdx
0x00000000004011c0 <+204>:   mov     %rdx,0x8(%rcx)
0x00000000004011c4 <+208>:   add     $0x8,%rax
0x00000000004011c8 <+212>:   cmp     %rsi,%rax
0x00000000004011cb <+215>:   je      0x4011d2 <phase_6+222>

```

```

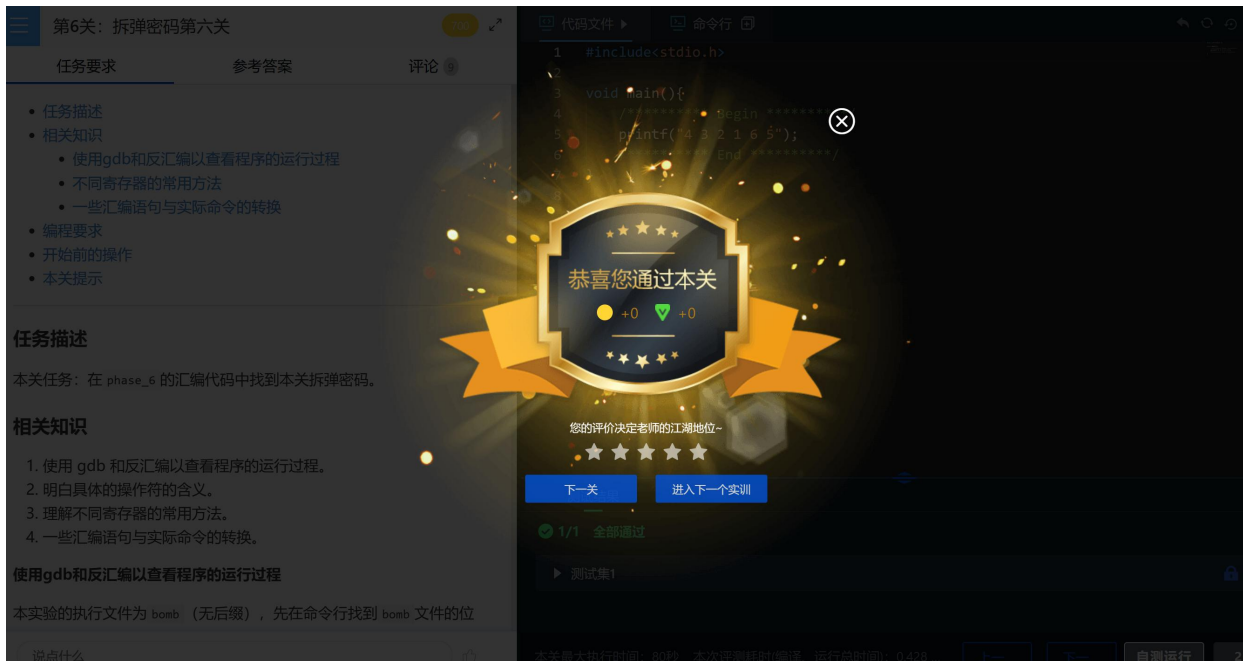
0x000000000004011c0 <+204>:  mov    %rdx,0x8(%rcx)
0x000000000004011c4 <+208>:  add     $0x8,%rax
0x000000000004011c8 <+212>:  cmp     %rsi,%rax
0x000000000004011cb <+215>:  je      0x4011d2 <phase_6+222>
0x000000000004011cd <+217>:  mov     %rdx,%rcx
---Type <return> to continue, or q <return> to quit---
0x000000000004011d0 <+220>:  jmp     0x4011bd <phase_6+201>
0x000000000004011d2 <+222>:  movq    $0x0,0x8(%rdx)
0x000000000004011da <+230>:  mov     $0x5,%ebp
0x000000000004011df <+235>:  mov     0x8(%rbx),%rax
0x000000000004011e3 <+239>:  mov     (%rax),%eax
0x000000000004011e5 <+241>:  cmp     %eax,(%rbx)
0x000000000004011e7 <+243>:  jge     0x4011ee <phase_6+250>
0x000000000004011e9 <+245>:  callq   0x40143a <explode_bomb>
0x000000000004011ee <+250>:  mov     0x8(%rbx),%rbx
0x000000000004011f2 <+254>:  sub     $0x1,%ebp
0x000000000004011f5 <+257>:  jne     0x4011df <phase_6+235>
0x000000000004011f7 <+259>:  add     $0x50,%rsp
0x000000000004011fb <+263>:  pop     %rbx
0x000000000004011fc <+264>:  pop     %rbp
0x000000000004011fd <+265>:  pop     %r12
0x000000000004011ff <+267>:  pop     %r13
0x00000000000401201 <+269>:  pop     %r14
0x00000000000401203 <+271>:  retq

```

● 实验过程：

1. 代码大致存在五个循环，其中具有嵌套解指针的结构，根据这个特征，得知这个代码是一个链表结构。
2. 可以根据代码重新编写成 c 语言代码(代码过长不再粘贴)。
3. 得知代码功能：输入一个 1-6 的数组（不能重复），对每个数字做 7 的被减数。根据数组的值，取到合适位置的链表节点，并由 ptr_array 存储。将 ptr_array 的元素按序排列，链表必须为降序。输入的数字为原链表位置，输入的次序为新链表的位置。因为第二个循环对每个数字做 7-array[i]，所以输入的数字需要将链表重排成为升序。原链表的顺序是 5 6 1 2 3 4，重排后是 4 3 2 1 6。
4. 得到最终结果为：“4 3 2 1 6”。

● 实验结果：



7. Phase7

● 实验代码：

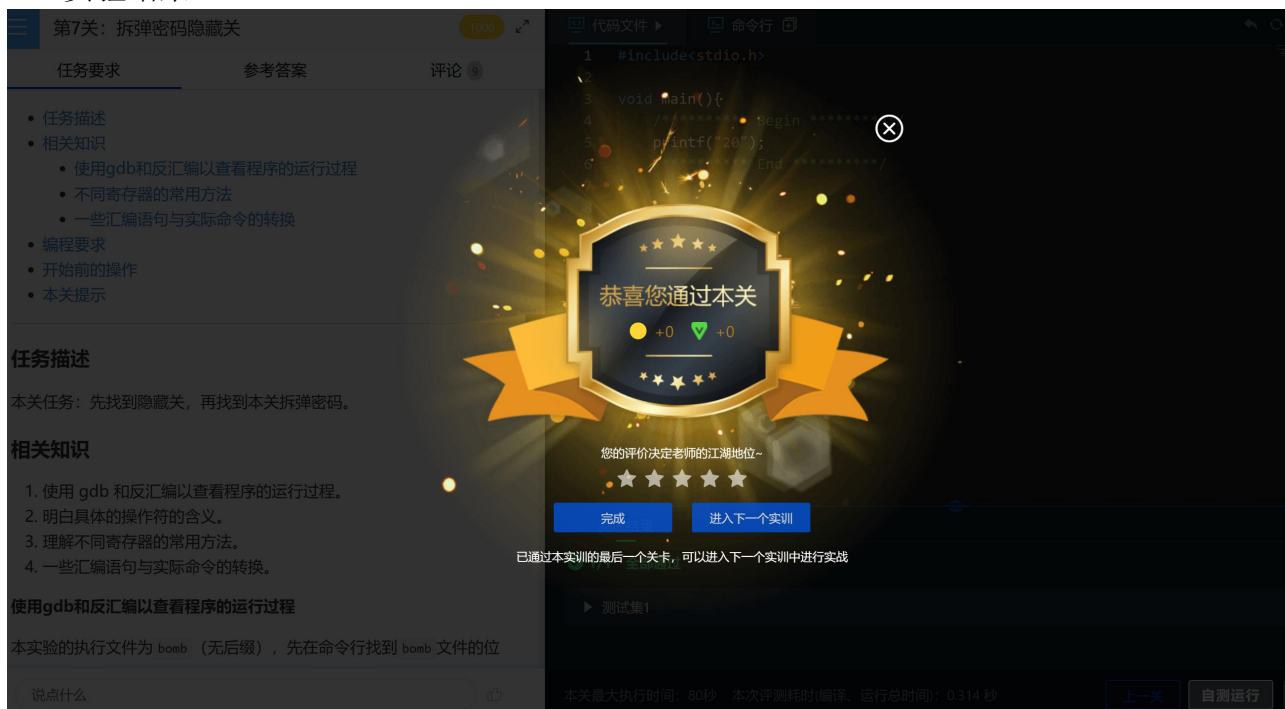
Dump of assembler code for function secret_phase:

```
0x0000000000401242 <+0>:      push    %rbx
0x0000000000401243 <+1>:      callq   0x40149e <read_line>
0x0000000000401248 <+6>:      mov     $0xa,%edx
0x000000000040124d <+11>:     mov     $0x0,%esi
0x0000000000401252 <+16>:     mov     %rax,%rdi
0x0000000000401255 <+19>:     callq   0x400bd0 <strtol@plt>
0x000000000040125a <+24>:     mov     %rax,%rbx
0x000000000040125d <+27>:     lea     -0x1(%rax),%eax
0x0000000000401260 <+30>:     cmp     $0x3e8,%eax
0x0000000000401265 <+35>:     jbe     0x40126c <secret_phase+42>
0x0000000000401267 <+37>:     callq   0x40143a <explode_bomb>
0x000000000040126c <+42>:     mov     %ebx,%esi
0x000000000040126e <+44>:     mov     $0x6030f0,%edi
0x0000000000401273 <+49>:     callq   0x401204 <fun7>
0x0000000000401278 <+54>:     cmp     $0x2,%eax
0x000000000040127b <+57>:     je      0x401282 <secret_phase+64>
0x000000000040127d <+59>:     callq   0x40143a <explode_bomb>
0x0000000000401282 <+64>:     mov     $0x402438,%edi
0x0000000000401287 <+69>:     callq   0x400b10 <puts@plt>
0x000000000040128c <+74>:     callq   0x4015c4 <phase_defused>
0x0000000000401291 <+79>:     pop     %rbx
0x0000000000401292 <+80>:     retq
```


● 实验过程:

1. 这是一个递归程序, %rdi、 %rdi+0x8 和%rdi+0x10 是一个指针, 这个数据结构就是二叉树了
2. 编写 c 语言代码(代码过长, 不再粘贴), 得需要一个整数传入 fun7 使返回值为 2, 则答案是 20。
3. 最终结果为: “20”。

● 实验结果:



四、实验总结

通过本次实验, 我的汇编语言能力得到了显著提升。在这个过程中, 我深入理解了一些关键的汇编语言概念, 如循环控制、跳转表等, 这些知识点在我的实践中得到了巩固和深化。实验中, 我遇到了多种汇编语言的实用技巧, 特别是跳转指令的灵活运用, 此外, 拆除炸弹的过程也锻炼了我的逻辑思维和问题解决能力。这个过程不仅提升了我的技术能力, 也增强了我的自信心。